

Diseño y Fragmentación de una Base de Datos Distribuida con PostgreSQL y Docker

Ivan Andres Villamarin
Aplicaciones Distribuidas
Universidad Técnica Estatal de Quevedo
Quevedo, Ecuador

Abstract— Este informe describe el diseño e implementación de una base de datos distribuida para un escenario académico de una cafetería universitaria con tres sedes: Campus, Babahoyo y Ventanas. Utilizando PostgreSQL 16 sobre contenedores Docker, se aplicaron técnicas de fragmentación horizontal, vertical y mixta sobre una base de datos centralizada, y se reconstruyó la información distribuida mediante vistas globales creadas con la extensión `postgres_fdw`. Los resultados de verificación confirman el cumplimiento de las propiedades de completitud, reconstrucción y disjunción, validando la correcta aplicación del diseño distribuido.

Index Terms— bases de datos distribuidas, fragmentación horizontal, fragmentación vertical, fragmentación mixta, PostgreSQL, Docker, `postgres_fdw`

I. Introducción

La presente práctica tiene como finalidad aplicar el diseño y la fragmentación de una base de datos distribuida utilizando PostgreSQL en contenedores Docker. Para el desarrollo se trabajó con un escenario académico basado en una cafetería universitaria que cuenta con tres sedes: Campus, Babahoyo y Ventanas.

El objetivo principal fue dividir la información de una base de datos centralizada en varios nodos independientes, aplicando fragmentación horizontal, fragmentación vertical y fragmentación mixta. Además, se implementaron vistas globales mediante `postgres_fdw` para reconstruir la información distribuida y comprobar que los datos pueden consultarse nuevamente como si estuvieran en una sola base de datos.

Esta práctica permite comprender de forma sencilla cómo una base de datos distribuida puede mejorar la organización de los datos, distribuir la carga entre varios nodos y mantener la posibilidad de reconstruir la información original mediante consultas SQL.

II. Objetivos

A. Objetivo general

Implementar una base de datos distribuida en PostgreSQL mediante contenedores Docker, aplicando fragmentación horizontal, vertical y mixta, y verificando la reconstrucción de los datos mediante vistas globales.

B. Objetivos específicos

- Crear un entorno distribuido con tres nodos PostgreSQL usando Docker Compose.
- Diseñar una base de datos centralizada para una cafetería universitaria.
- Aplicar fragmentación horizontal sobre la tabla pedidos.
- Aplicar fragmentación vertical sobre la tabla clientes.
- Aplicar fragmentación mixta combinando criterios de ciudad y tipo de dato.
- Crear vistas globales usando `postgres_fdw`.
- Verificar las condiciones de completitud, reconstrucción y disjunción.

III. Entorno de Trabajo

Para el desarrollo de la práctica se utilizaron las siguientes herramientas:

- Visual Studio Code como editor de código.
- Docker Desktop para ejecutar los contenedores.
- Docker Compose para definir y levantar los servicios.
- PostgreSQL 16 como sistema gestor de base de datos.
- PowerShell como terminal de ejecución.
- `postgres_fdw` como extensión para conectar tablas remotas.

La estructura del proyecto fue la siguiente:

```
practica-fragmentacion/  
  docker-compose.yml  
  README.md  
  sql/  
    01_esquema_central.sql  
    02_datos.sql  
    03_fragmentacion_horizontal.sql  
    04_fragmentacion_vertical.sql  
    05_fragmentacion_mixta.sql  
    06_vistas_globales.sql  
    07_verificacion.sql
```

Se configuraron tres contenedores PostgreSQL, cada uno representando una sede diferente: pg-campus (sede Campus), pg-babahoyo (sede Babahoyo) y pg-ventanas (sede Ventanas). Cada nodo trabaja como una instancia independiente de PostgreSQL, lo que permite simular una base de datos distribuida sobre una sola computadora.

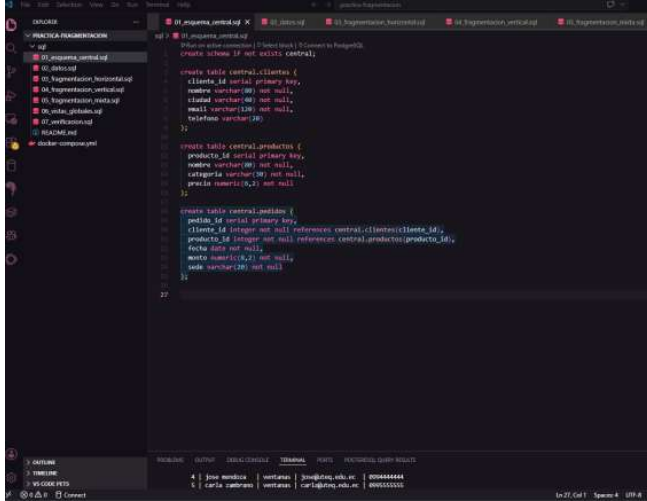


Figure 1: Estructura del proyecto en Visual Studio Code, mostrando los scripts SQL y el archivo docker-compose.yml.

IV. Diseño de la Base de Datos Centralizada

Antes de aplicar la fragmentación, se creó una base de datos centralizada llamada *cafeteria*. Esta base contiene tres tablas principales: *clientes*, *productos* y *pedidos*.

La tabla *clientes* almacena los datos de los estudiantes o usuarios registrados en la cafetería. Sus atributos son *cliente_id*, *nombre*, *ciudad*, *email* y *telefono*.

La tabla *productos* registra los productos disponibles, como café, sándwiches, jugos y empanadas. Sus atributos son *producto_id*, *nombre*, *categoría* y *precio*.

La tabla *pedidos* representa las compras realizadas por los clientes. Esta tabla se relaciona con *clientes* y *productos* mediante claves foráneas. Sus atributos son *pedido_id*, *cliente_id*, *producto_id*, *fecha*, *monto* y *sede*.

El modelo utilizado permite representar una relación donde un cliente puede realizar varios pedidos y cada pedido pertenece a un producto. La columna *sede* fue fundamental para realizar la fragmentación horizontal, ya que permite identificar en qué sede se generó cada pedido.

V. Fragmentación Horizontal

La fragmentación horizontal consiste en dividir una tabla por filas. En esta práctica se aplicó sobre la tabla *pedidos*, usando como criterio la columna *sede*.

Cada nodo almacena solamente los pedidos que corresponden a su sede: el nodo pg-campus almacena los pedidos de la sede Campus, el nodo pg-babahoyo almacena los pedidos de la sede Babahoyo, y el nodo pg-ventanas almacena los pedidos de la sede Ventanas.

Table 1: Resultados de la fragmentación horizontal

Nodo	Sede	N.º de pedidos
pg-campus	campus	3
pg-babahoyo	babahoyo	3
pg-ventanas	ventanas	2

Esta fragmentación es correcta porque cada pedido pertenece a una sola sede. Por lo tanto, no existen pedidos duplicados entre los nodos. Además, al unir los fragmentos de los tres nodos se obtiene nuevamente el total de 8 pedidos originales.

La fragmentación horizontal permite distribuir la carga de datos entre varias sedes. En un sistema real, esto podría ayudar a que cada sede consulte sus propios pedidos sin depender totalmente de un servidor central.

VI. Fragmentación Vertical

La fragmentación vertical consiste en dividir una tabla por columnas. En esta práctica se aplicó sobre la tabla *clientes*, que fue separada en dos partes: *clientes_publicos*, que contiene *cliente_id*, *nombre* y *ciudad*; y *clientes_contacto*, que contiene *cliente_id*, *email* y *telefono*.

Table 2: Resultados de la fragmentación vertical

Nodo	Tabla	Contenido
pg-campus	clientes_publicos	Datos generales del cliente
pg-babahoyo	clientes_contacto	Datos de contacto del cliente

La clave primaria *cliente_id* se conserva en ambos fragmentos. Esto es necesario porque permite reconstruir la tabla original mediante una operación *JOIN*.

Esta fragmentación puede ser útil cuando se desea separar información pública de información sensible. Por ejemplo, el nombre y la ciudad pueden estar disponibles para consultas generales, mientras que el correo y el teléfono pueden protegerse en otro nodo con mayor control de acceso.

VII. Fragmentación Mixta

La fragmentación mixta combina fragmentación horizontal y vertical. En esta práctica se aplicó sobre los datos de clientes. Primero se separaron los clientes según la ciudad y luego se separaron los datos según su tipo, obteniendo los siguientes fragmentos.

El nodo pg-campus almacena los datos públicos de los clientes de Quevedo. El nodo pg-ventanas almacena los

Table 3: Resultados de la fragmentación mixta

Nodo	Tabla	Contenido
pg-campus	clientes_quevedo_publicos	Clientes de Quevedo con datos públicos
pg-babahoyo	clientes_contacto_mixto	Datos de contacto de todos los clientes
pg-ventanas	clientes_otros_publicos	Clientes de Babahoyo y Ventanas con datos públicos

datos públicos de los clientes de Babahoyo y Ventanas. El nodo pg-babahoyo almacena los datos de contacto de todos los clientes.

Esta fragmentación demuestra que en un sistema distribuido se pueden combinar varios criterios de división. Por un lado, se puede distribuir la información por ubicación geográfica, y por otro lado, se puede separar la información sensible de la información general.

VIII. Vistas Globales con postgres_fdw

Para reconstruir los datos distribuidos se utilizó la extensión postgres_fdw de PostgreSQL. Esta extensión permite que un nodo consulte tablas ubicadas en otros nodos como si fueran tablas locales.

La configuración se realizó desde el nodo pg-campus. Desde este nodo se crearon conexiones hacia pg-babahoyo y pg-ventanas. Se crearon dos vistas globales principales: pedidos_global y clientes_global.

La vista pedidos_global reconstruye los pedidos de los tres nodos mediante UNION ALL, uniendo los pedidos almacenados en pg-campus, pg-babahoyo y pg-ventanas.

La vista clientes_global reconstruye los datos completos de los clientes mediante JOIN, utilizando cliente_id como clave de unión entre clientes_publicos y clientes_contacto.

Los resultados principales fueron: la vista pedidos_global mostró 8 pedidos, la vista clientes_global mostró 6 clientes completos, y la consulta de total por sede devolvió los valores correctos.

Esto demuestra que, aunque los datos están distribuidos físicamente en varios contenedores, pueden ser consultados de manera integrada mediante vistas globales.

IX. Verificación de Resultados

Para comprobar que la fragmentación fue correcta, se aplicaron tres condiciones principales: completitud, reconstrucción y disjunción.

A. Completitud

La completitud significa que ningún dato debe perderse durante la fragmentación. Para comprobarlo, se contó el número

```
cafeteria=# SELECT * FROM pedidos_global;
```

pedido_id	cliente_id	producto_id	fecha	monto	sede
1	1	1	2026-03-01	0.75	campus
3	3	2	2026-03-02	1.00	campus
7	1	3	2026-03-04	2.50	campus
2	2	3	2026-03-01	2.50	babahoyo
6	6	1	2026-03-03	0.75	babahoyo
8	2	2	2026-03-04	1.00	babahoyo
4	4	4	2026-03-02	1.25	ventanas
5	5	5	2026-03-03	1.00	ventanas

(8 rows)

Figure 2: Consulta SELECT * FROM pedidos_global mostrando los 8 pedidos reconstruidos de las tres sedes.

```
cafeteria=# SELECT * FROM clientes_global;
```

cliente_id	nombre	ciudad	email	telefono
1	maria alvarado	quevedo	maria@uteq.edu.ec	0991111111
2	luis ceden	babahoyo	luis@uteq.edu.ec	0992222222
3	ana vera	quevedo	ana@uteq.edu.ec	0993333333
4	jose mendoza	ventanas	jose@uteq.edu.ec	0994444444
5	carla zambrano	ventanas	carla@uteq.edu.ec	0995555555
6	pedro suarez	babahoyo	pedro@uteq.edu.ec	0996666666

(6 rows)

Figure 3: Consulta SELECT * FROM clientes_global mostrando los 6 clientes reconstruidos mediante JOIN.

de filas en la vista pedidos_global, obteniendo un resultado de filas_globales = 8. Este resultado confirma que los 8 pedidos originales se encuentran distribuidos entre los tres nodos.

B. Reconstrucción

La reconstrucción significa que la información original puede volver a obtenerse a partir de los fragmentos. Para verificarlo, se consultó el total vendido por sede desde la vista pedidos_global.

Table 4: Total vendido por sede

Sede	Total
babahoyo	4.25
ventanas	2.25
campus	4.25

También se verificó la reconstrucción vertical mediante la vista clientes_global, obteniendo un resultado de clientes_reconstruidos = 6.

C. Disjunción

La disjunción significa que los datos no deben repetirse innecesariamente en varios fragmentos. Para comprobarlo, se buscó si algún pedido_id aparecía más de una vez, obteniendo un resultado de 0 filas. Esto confirma que ningún pedido está duplicado entre los nodos.

```

PS C:\Users\Labo_0.02\Music\practica-fragmentacion> Get-Content .\sql\07_verificacion.sql
1 | docker exec -i pg-campus psql -U admin -d cafeteria
    filas_globales
-----
      8
(1 row)

    sede | total
-----+-----
babahoyo | 4.25
ventanas | 2.25
campus   | 4.25
(3 rows)

 pedido_id | veces
-----+-----
(0 rows)

 clientes_reconstruidos
-----
      6
(1 row)

 cliente_id | nombre | ciudad | email | telefono
-----+-----+-----+-----+-----
1 | maria alvarado | quevedo | maria@uteq.edu.ec | 0991111111
2 | luis cedeno | babahoyo | luis@uteq.edu.ec | 0992222222
3 | ana vera | quevedo | ana@uteq.edu.ec | 0993333333
4 | jose mendoza | ventanas | jose@uteq.edu.ec | 0994444444
5 | carla zambrano | ventanas | carla@uteq.edu.ec | 0995555555
6 | pedro suarez | babahoyo | pedro@uteq.edu.ec | 0996666666
(6 rows)

PS C:\Users\Labo_0.02\Music\practica-fragmentacion>

```

Figure 4: Verificación final de completitud, reconstrucción y disjunción ejecutada desde PowerShell mediante `docker exec`.

X. Reflexión

La práctica permitió comprender que una base de datos distribuida no consiste solamente en tener varias copias de una misma base, sino en organizar los datos de forma estratégica entre varios nodos. La fragmentación horizontal resultó fácil de entender porque separa los registros según una condición clara, en este caso la sede. La fragmentación vertical fue útil para observar cómo se pueden separar datos públicos y datos sensibles. La fragmentación mixta mostró que en un sistema real se pueden combinar varios criterios de distribución.

Una dificultad encontrada fue la ejecución de scripts desde PowerShell, ya que el operador de redirección menor-que no funciona igual que en CMD o Linux. Para solucionarlo se utilizó `Get-Content` junto con `docker exec`. Otra dificultad fue comprender que algunos scripts debían ejecutarse por partes en nodos diferentes; esto se resolvió comentando y descomentando los bloques correspondientes según el nodo donde se ejecutaba.

En general, la práctica ayudó a reforzar los conceptos de completitud, reconstrucción y disjunción. Estas tres condiciones son importantes porque permiten comprobar que la fragmentación no haya provocado pérdida de datos, duplicación incorrecta o imposibilidad de reconstruir la información original.

XI. Conclusión

En conclusión, se logró implementar una base de datos distribuida usando PostgreSQL y Docker, aplicando fragmentación horizontal, vertical y mixta. La tabla `pedidos` fue distribuida por sede, la tabla `clientes` fue separada

por tipo de información y posteriormente se combinó la distribución por ciudad y sensibilidad de los datos. Mediante `postgres_fdw` se crearon vistas globales que permitieron reconstruir la información distribuida. Los resultados de verificación confirmaron que se cumplió con la completitud, la reconstrucción y la disjunción, por lo que la fragmentación aplicada fue correcta.

Repositorio

El código fuente completo de la práctica, incluyendo los archivos `docker-compose.yml` y los scripts SQL, se encuentra disponible en el siguiente repositorio de GitHub:

<https://github.com/ivillamarinc/practica-fragmentacion>